As a strict grader and senior Go/ML engineer, I have reviewed **Feature Spec 001** (spec.md) against the project **Constitution** (constitution.md), the **Assignment Rubric** (assignment-and-rubric.md), and the empirical findings in **Spike 001** (001-gomlx-onnx.md). While the spec is exceptionally thorough and well-structured, a strict grading rubric reveals several critical ambiguities, missing implementation details, untestable assertions, and high-risk acceptance criteria that must be resolved before proceeding to plan.md.

1. Ambiguities and Architectural Risks

A. Unresolved Model and Framework Scope (spec.md Section 4 [FR-1], Section 11 [Q1, Q2])

- **The Issue:** The spec is currently tagged as DRAFT v1 — awaiting Gemini review and Luis's answers to section 11. Section 4 links requirements FR-1 and FR-2 to open questions Q1 (PyTorch vs scikit-learn) and Q2 (MNIST vs synthetic/tabular data).
- **The Risk:** Spike 001 explicitly validated a scikit-learn MLPClassifier with synthetic tabular data. Section 11 recommends PyTorch and MNIST, but PyTorch ONNX export (torch.onnx.export) and tensor preprocessing (channel dimensions, float32 normalization, flattening 28x28 images) introduce completely different data pipelines than tabular sklearn models. Leaving this undecided in the spec makes downstream functional requirements ambiguous.

B. Cross-Compilation & Backend Mismatch (spec.md Section 6 [Finding 5], Section 8 [AC-8])

- **The Issue:** Spike 001 highlights that GoMLX relies on two distinct backends: XLA (fast, C-dependent, macOS/Linux) and pure-Go (portable, zero C-dependencies). Section 8 specifies AC-8: *"Build commands produce a macOS binary, a Windows .exe, and a macOS .app"*.
- **The Risk:** Cross-compiling a Go application that links GoMLX/XLA C-bindings for Windows (GOOS=windows GOARCH=amd64) will fail unless it forces the pure-Go backend (GOMLX_BACKEND=go). The spec mandates benchmarking *both* backends on "the same machine" (FR-6), but a Windows .exe compiled with pure-Go will yield drastically different latency and throughput profiles than macOS running XLA. A strict grader will flag this as an apples-to-oranges performance comparison.

C. Graph Compilation vs. Model Loading Isolation (spec.md Section 4 [FR-7])

- **The Issue:** FR-7 requires reporting start-up cost separately, noting that *"GoMLX compiles a graph on first use"*.
- **The Risk:** GoMLX lazy-compiles graphs during the first CallOnce or inference execution, not during parser.ParseFile. If "start-up cost" is measured by simply timing the file load or initialization function, it will omit graph compilation time. The spec fails to define the exact program boundary or hook for measuring first-inference compilation overhead.

2. Missing Requirements and Testability Gaps

A. Absence of Explicit NaN/Inf Validation in Parity Checks (constitution.md Article 3 [3.3])

- **The Issue:** Constitution Article 3.3 mandates that *"Floating-point comparisons use a named tolerance constant, and reject NaN explicitly"*. While FR-12 covers input validation for non-finite values, neither FR-4 nor the parity verification requirements mandate checking model output probability tensors for NaN or Inf values resulting from numerical instability or invalid ONNX weight translations.

B. Unspecified Profiling Artifacts (spec.md Section 7 [NFR-3], Section 8 [AC-5])

- **The Issue:** NFR-3 and AC-5 require that *"CPU and memory profiles of Go inference are generated and kept"* and *"summarized"*.
- **The Risk:** Without specifying the exact file format (.prof, flamegraphs, or text dumps), directory paths (results/profiles/), or tooling (go tool pprof), different implementations could produce untestable or non-standard profiling artifacts, violating Constitution Article 3.4 (*"Benchmarks use testing.B with allocation reporting; CPU and memory profiles are generated and kept"*).

C. Manual README Table Generation Risk (spec.md Section 4 [FR-13], constitution.md Article 1 [1.3])

- **The Issue:** AC-9 states that the README contains tables generated from results/.
- **The Risk:** Constitution Article 1.3 states: *"Every numeric claim in the README is reproducible by a command a grader can run, and is copied from program output — never hand-typed"*. The spec does not mandate a script or Makefile target (e.g., make report) to automatically inject benchmark results into README.md, leaving it vulnerable to manual copy-paste errors.

3. Acceptance Criteria a Strict Grader Will Reject or Mark as "Not Met"

A strict grader evaluating the spec and rubric constraints (assignment-and-rubric.md) would challenge the following Acceptance Criteria:

1. **AC-8 ("Build commands produce a macOS binary, a Windows .exe, and a macOS .app")**
   - *Grader Verdict:* **Not Testable / High Risk of Failure.** Cross-compiling a GoMLX application for Windows without clear instructions on handling CGo/XLA dependencies vs. pure-Go fallback will break on standard CI/CD or grader machines unless explicitly defined in plan.md. Furthermore, a .app bundle requires a specific directory structure (Contents/MacOS/, Info.plist), which is entirely missing from the spec design.
2. **AC-4 ("The Monte Carlo benchmark reports time and memory for batch sizes 1 to 10,000")**
   - *Grader Verdict:* **Conflict with Constitution 3.4.** Constitution Article 3.4 requires benchmarks to use testing.B with allocation reporting. If AC-4 is implemented as a standalone CLI command (go run ./cmd/benchmark) rather than standard go test

-bench, it fails Constitution compliance for benchmarking rigor.

3. **AC-6 ("Whole-program wall time and peak memory are reported for Python and Go")**
    - *Grader Verdict:* **Ambiguous Measurement Methodology.** The spec does not specify *how* peak memory is measured across operating systems (e.g., using Unix time -v vs. Go's runtime.MemStats vs. Python's tracemalloc). Without a standardized measurement script, Python and Go measurements will be inconsistent.

Required Remediation Before Planning

Per Constitution Article 7.4 (*"If implementation reveals the spec is wrong, update the spec first, then the code"*), Luis must resolve **Section 11 (Q1 and Q2)** and update spec.md to lock in either PyTorch/MNIST or scikit-learn/synthetic data, specify the exact profiling output paths for AC-5, and clarify cross-compilation targets for AC-8 before writing plan.md.